

1. Case-Based Learning vs Locally Weighted Regression

```
# =====
# CASE-BASED LEARNING vs LOCALLY WEIGHTED REGRESSION
# Practical Example: Data Plan Recommendation
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# -----
# 1. Create Dataset
# -----

data = pd.DataFrame({
    'data_usage_gb': [2, 4, 5, 7, 9, 10, 12, 15, 18, 20, 23, 25],
    'call_minutes': [100, 150, 180, 220, 250, 280, 300, 350, 400, 450, 500, 550],
    'plan_price': [199, 249, 299, 349, 399, 399, 449, 499, 549, 599, 649, 699]
})

print("Dataset:")
print(data)

# -----
# 2. Case-Based Learning
# -----

def case_based_learning(query_usage, query_calls, k=3):

    # Calculate normalized distance
    usage_range = data['data_usage_gb'].max() - data['data_usage_gb'].min()
    call_range = data['call_minutes'].max() - data['call_minutes'].min()

    distances = np.sqrt(
        ((data['data_usage_gb'] - query_usage) / usage_range) ** 2 +
        ((data['call_minutes'] - query_calls) / call_range) ** 2
    )

    # Find k most similar cases
    nearest_indices = distances.nsmallest(k).index

    nearest_cases = data.loc[nearest_indices]

    # Average price of similar cases
    recommendation = nearest_cases['plan_price'].mean()

    return recommendation, nearest_cases
```

```

# -----
# 3. Locally Weighted Regression
# -----

def locally_weighted_regression(x_query, bandwidth=3):

    x = data['data_usage_gb'].values
    y = data['plan_price'].values

    # Calculate weights
    weights = np.exp(-((x - x_query) ** 2) / (2 * bandwidth ** 2))

    # Design matrix for linear regression
    X = np.vstack([np.ones(len(x)), x]).T

    # Weighted least squares
    W = np.diag(weights)

    theta = np.linalg.pinv(X.T @ W @ X) @ X.T @ W @ y

    prediction = theta[0] + theta[1] * x_query

    return prediction

# -----
# 4. New Customer
# -----

new_usage = 11
new_calls = 290

print("\nNew Customer:")
print("Data Usage:", new_usage, "GB")
print("Call Minutes:", new_calls)

# Case-Based Learning
cbl_prediction, similar_cases = case_based_learning(
    new_usage,
    new_calls,
    k=3
)

print("\n--- Case-Based Learning ---")
print("Similar Cases:")
print(similar_cases)
print("Recommended Plan Price: ₹", round(cbl_prediction, 2))

# LWR

```

```

lwr_prediction = locally_weighted_regression(new_usage)

print("\n--- Locally Weighted Regression ---")
print("Predicted Plan Price: ₹", round(lwr_prediction, 2))

# -----
# 5. Compare Predictions
# -----

print("\n--- Comparison ---")
print("CBL Recommendation : ₹", round(cbl_prediction, 2))
print("LWR Prediction      : ₹", round(lwr_prediction, 2))

# -----
# 6. Plot
# -----

usage_values = np.linspace(
    data['data_usage_gb'].min(),
    data['data_usage_gb'].max(),
    100
)

lwr_values = [
    locally_weighted_regression(x)
    for x in usage_values
]

plt.figure(figsize=(10, 6))

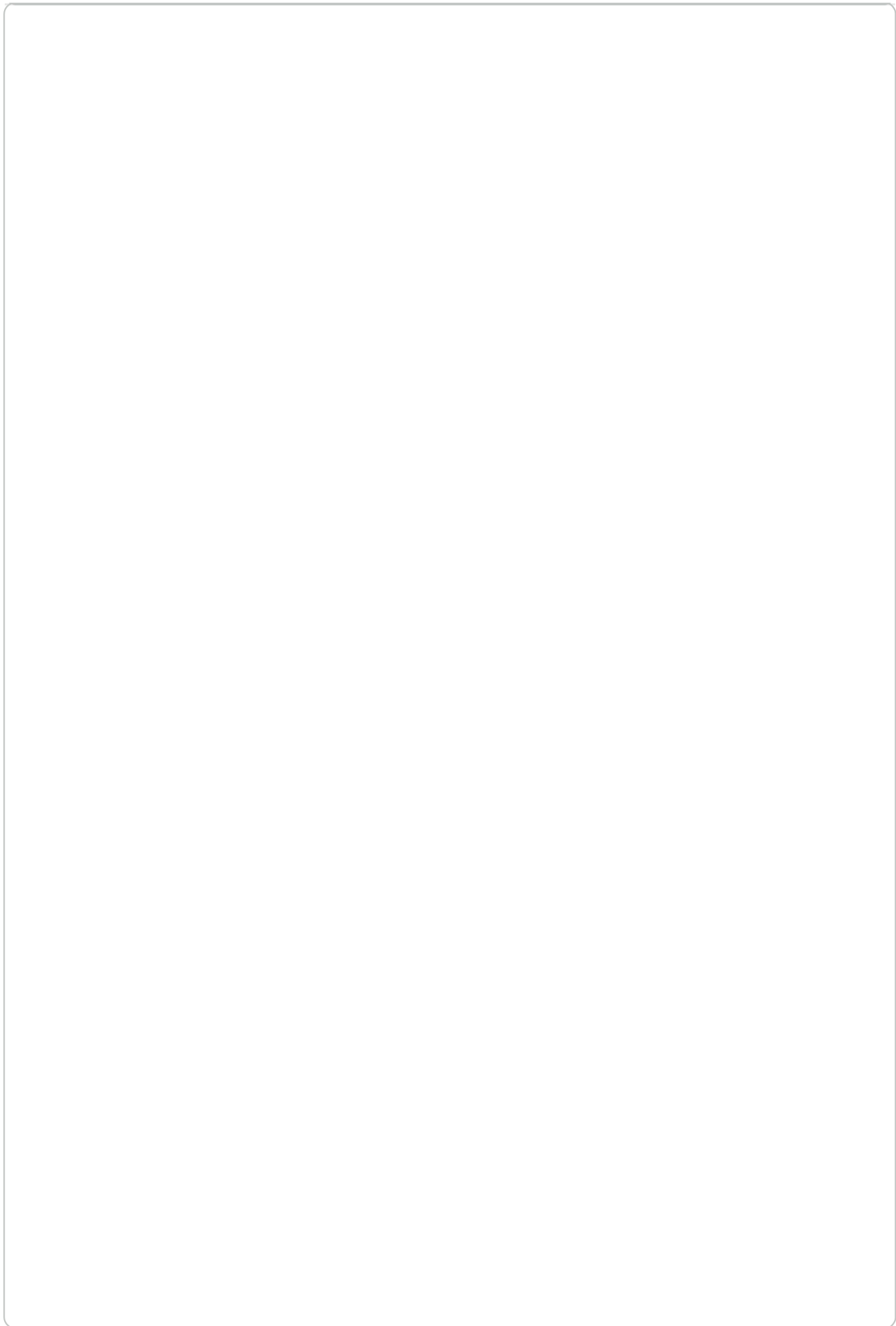
plt.scatter(
    data['data_usage_gb'],
    data['plan_price'],
    label='Training Data'
)

plt.plot(
    usage_values,
    lwr_values,
    label='Locally Weighted Regression'
)

plt.scatter(
    new_usage,
    lwr_prediction,
    s=100,
    label='New Customer'
)

```

```
plt.xlabel("Data Usage (GB)")  
plt.ylabel("Plan Price (₹)")  
plt.title("CBL vs Locally Weighted Regression")  
plt.legend()  
plt.grid(True)  
plt.show()
```



Dataset:

	data.usage.gb	call.minutes	plan.price
0	2	100	199
1	4	150	249

2. Locally Weighted Regression vs RBF Network

```
# =====
# LOCALLY WEIGHTED REGRESSION vs RBF NETWORK
# Function Approximation using  $y = \sin(x)$ 
# =====

import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Create Dataset
# -----

np.random.seed(42)

X = np.linspace(0, 10, 30)

# Generate noisy sine data
y = np.sin(X) + np.random.normal(0, 0.1, len(X))

print("Sample Dataset:")
for i in range(10):
    print("x =", round(X[i], 2),
          " y =", round(y[i], 3))

# -----
# 2. Locally Weighted Regression
# -----

def locally_weighted_regression(x_query, bandwidth=0.6):

    # Calculate weights
    weights = np.exp(
        -((X - x_query) ** 2) /
        (2 * bandwidth ** 2)
    )

    # Local linear regression
    A = np.vstack([
        np.ones(len(X)),
        X
    ]).T

    W = np.diag(weights)

    theta = np.linalg.pinv(
        A.T @ W @ A
```

```

    ) @ A.T @ W @ y

    prediction = theta[0] + theta[1] * x_query

    return prediction

# -----
# 3. RBF Network
# -----

# Select RBF centers
centers = np.linspace(0, 10, 10)

# Width of Gaussian functions
sigma = 1.0

def rbf_function(x, center, sigma):
    return np.exp(
        -((x - center) ** 2) /
        (2 * sigma ** 2)
    )

# Create RBF feature matrix
Phi = np.zeros((len(X), len(centers)))

for i in range(len(X)):
    for j in range(len(centers)):
        Phi[i, j] = rbf_function(
            X[i],
            centers[j],
            sigma
        )

# Calculate output weights
weights = np.linalg.pinv(Phi) @ y

# -----
# 4. RBF Prediction Function
# -----

def rbf_predict(x):

    phi = np.array([
        rbf_function(x, c, sigma)
        for c in centers
    ])

```

```
    return np.dot(phi, weights)

# -----
# 5. Generate Predictions
# -----

X_test = np.linspace(0, 10, 200)

lwr_predictions = np.array([
    locally_weighted_regression(x)
    for x in X_test
])

rbf_predictions = np.array([
    rbf_predict(x)
    for x in X_test
])

# -----
# 6. Plot LWR
# -----

plt.figure(figsize=(10, 6))

plt.scatter(
    X,
    y,
    label='Training Data'
)

plt.plot(
    X_test,
    lwr_predictions,
    label='LWR Prediction'
)

plt.plot(
    X_test,
    np.sin(X_test),
    linestyle='--',
    label='Actual sin(x)'
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Locally Weighted Regression")
plt.legend()
plt.grid(True)
plt.show()
```



```
# -----  
# 7. Plot RBF  
# -----  
  
plt.figure(figsize=(10, 6))  
  
plt.scatter(  
    X,  
    y,  
    label='Training Data'  
)  
  
plt.plot(  
    X_test,  
    rbf_predictions,  
    label='RBF Network'  
)  
  
plt.plot(  
    X_test,  
    np.sin(X_test),  
    linestyle='--',  
    label='Actual sin(x)'  
)  
  
plt.xlabel("X")  
plt.ylabel("Y")  
plt.title("Radial Basis Function Network")  
plt.legend()  
plt.grid(True)  
plt.show()  
  
# -----  
# 8. Compare Both  
# -----  
  
plt.figure(figsize=(10, 6))  
  
plt.scatter(  
    X,  
    y,  
    label='Training Data'  
)  
  
plt.plot(  
    X_test,  
    lwr_predictions,  
    label='LWR'  
)
```

```
plt.plot(
    X_test,
    rbf_predictions,
    label='RBF Network'
)

plt.plot(
    X_test,
    np.sin(X_test),
    linestyle='--',
    label='Actual sin(x)'
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("LWR vs RBF Network")
plt.legend()
plt.grid(True)
plt.show()

# -----
# 9. Calculate Mean Squared Error
# -----

actual = np.sin(X_test)

lwr_mse = np.mean(
    (actual - lwr_predictions) ** 2
)

rbf_mse = np.mean(
    (actual - rbf_predictions) ** 2
)

print("\n--- Performance Comparison ---")

print("LWR Mean Squared Error :",
      round(lwr_mse, 4))

print("RBF Mean Squared Error :",
      round(rbf_mse, 4))

if lwr_mse < rbf_mse:
    print("\nLWR gives lower error for this dataset.")
else:
    print("\nRBF Network gives lower error for this dataset.")
```

